EECS 442 HW5: Panorama

Please provide the following information (e.g. John Doe, jdoe):

[Your first name] [Your last name], [Your UMich uniqname]

Overview

In this homework, you will implement panoramic stitching. Given some input images of the same static scene, we will "stitch" them together to create a single panorama. Our framework consists of the following main steps:

1. Keypoint & feature extraction using SIFT
2. Correspondence matching
3. Homography estimation (with RANSAC for robustness)
4. Perspective warping

We will primarily focus on homography estimation in Problem 1 using a toy example, then implement the entire framework and apply it to real-world images in Problem 2.

Submission instructions: This homework will be done entirely through a Google Colab notebook. When finished, please submit to Gradescope **only** a `.pdf` file of your completed notebook file. You can use `Ctrl/Cmd + P` or the code block at the end of this notebook, but make sure the pdf contains **BOTH your code and all required visualizations**.

Late policy: As stated in the syllabus, submissions are accepted with full credit decreasing linearly to zero over 5 days, at the rate of 1 point per hour, from 120 points on the due date at 5:29 pm to 0 points 120 hours after the due date. No exceptions to this policy are to be given.

Problem 1: Homography Estimation

First, we will implement a function that computes a homography matrix H from a set of corresponding points between two images. We'll estimate H using two methods: 1) an analytic (linear) method and 2) a nonlinear least squares optimization method. For a pair of corresponding points, p_i and p'_i, we solve for a homography matrix H such that $p'_i = Hp_i$:

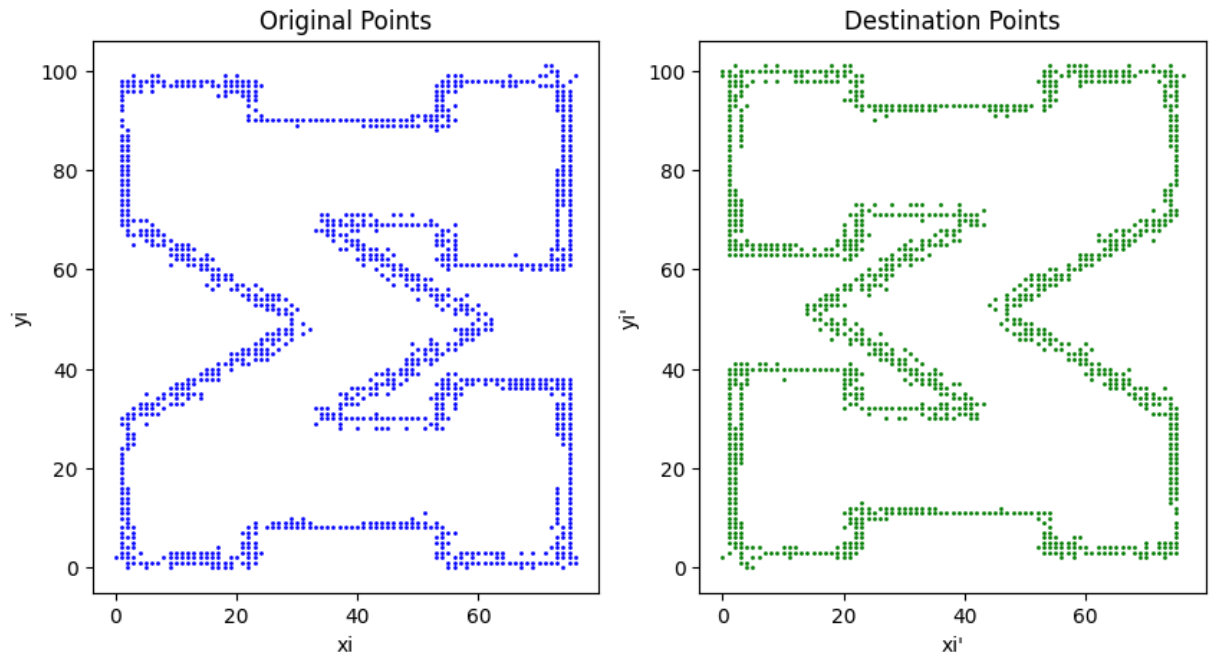
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

We will work with two sets of 2D points representing the shape of the University of Michigan's "M" logo, but at different orientations (see below for visualization). One set serves as the original points, while the other represents the destination points. Our goal is to compute the homography that maps the original points to the destination points, enabling us to warp the original shape accordingly.

In [11]:
```
%%capture
!wget -O file1.npy 'https://docs.google.com/uc?export=download&id=1c4i4gsSRs
```

In [12]:
```
import numpy as np
import matplotlib.pyplot as plt

data = np.load("file1.npy")
original_points, destination_points = data[:, 0:2], data[:, 2:]

print('original points: ', original_points.shape)
print('destination points: ', destination_points.shape)

fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].scatter(original_points[:, 0], original_points[:, 1], s=1, c='blue')
ax[0].set_title('Original Points')
ax[0].set_xlabel('xi')
ax[0].set_ylabel('yi')
ax[1].scatter(destination_points[:, 0], destination_points[:, 1], s=1, c='gr
ax[1].set_title('Destination Points')
ax[1].set_xlabel("xi'")
ax[1].set_ylabel("yi'")
plt.show()
```

```
original points:  (1205, 2)
destination points:  (1205, 2)
```



(a) For a pair of corresponding points p_i and p_i', obtain an analytic solution for H.

x' = (ax + by + c) / (gx + hy + i), y' = (dx + ey + f) / (gx + hy + i)

Multiplying both sides by the denominator, we obtain two constraints:

Constraint 1: [-x, -y, -1, 0, 0, 0, x·x', y·x', x'] · [a,b,c,d,e,f,g,h,i]^T = 0

Constraint 2:[0, 0, 0, -x, -y, -1, x·y', y·y', y'] · [a,b,c,d,e,f,g,h,i]^T = 0

Each pair of corresponding points provides 2 linear constraints. Since H has 9 unknowns but is defined only up to scale (8 degrees of freedom), at least 4 point correspondences (providing 8 constraints) are required to fully determine H.

(b) Implement the following function that computes the homography matrix from a set of point correspondences using the **analytic solution**.

Hint: You may use `np.linalg.svd`. Be sure to normalize H so that $i = 1$.

In [13]:
```
def fit_homography(pts1 ,pts2):
    """
    Given a set of N correspondences of the form [x,y], [x',y'],
    fit a homography that maps [x,y,1] to [x',y',1] using the analytic solut

    Input:
        pts1, pts2: (N, 2) matrices representing N corresponding points [x,
    Returns:
        H: a (3,3) homography matrix that satisfies [x',y',1] == H [x,y,1]
```

```
    """

    H = None

    ##########################################################################
    #                                    TODO
    ##########################################################################
    if pts1.shape != pts2.shape:
        raise ValueError('pts1 and pts2 must have the same shape')
    if pts1.shape[0] < 4:
        raise ValueError('At least 4 point correspondences are required')

    A = []
    for (x, y), (xp, yp) in zip(pts1, pts2):
        A.append([x, y, 1, 0, 0, 0, -x * xp, -y * xp, -xp])
        A.append([0, 0, 0, x, y, 1, -x * yp, -y * yp, -yp])

    A = np.asarray(A, dtype=float)
    _, _, vh = np.linalg.svd(A)
    H = vh[-1].reshape(3, 3)
    H = H / H[2, 2]
    ##########################################################################
    #                              END OF YOUR CODE
    ##########################################################################

    return H
```

(c) Implement the following function that computes the homography matrix from a set of point correspondences using **nonlinear least squares optimization**.

Hint: You may use `scipy.optimizeleast_squares`.

In [14]:
```
def find_homography_nls(pts_1, pts_2):
    """
    Given a set of N correspondences of the form [x,y], [x',y'],
    fit a homography that maps [x,y,1] to [x',y',1] using nonlinear least sc

    Input:
        pts1, pts2: (N, 2) matrices representing N corresponding points [x,
    Returns:
        H: a (3,3) homography matrix that satisfies [x',y',1] == H [x,y,1]
    """

    from scipy.optimize import least_squares as nls

    H = None

    ##########################################################################
    #                                    TODO
    ##########################################################################
    if pts_1.shape != pts_2.shape:
        raise ValueError('pts_1 and pts_2 must have the same shape')
    if pts_1.shape[0] < 4:
```

```
        raise ValueError('At least 4 point correspondences are required')

    pts_1_h = np.hstack([pts_1, np.ones((pts_1.shape[0], 1))])
    H_init = fit_homography(pts_1, pts_2)
    x0 = np.array([
        H_init[0, 0], H_init[0, 1], H_init[0, 2],
        H_init[1, 0], H_init[1, 1], H_init[1, 2],
        H_init[2, 0], H_init[2, 1],
    ])

    def residuals(params):
        H_curr = np.array([
            [params[0], params[1], params[2]],
            [params[3], params[4], params[5]],
            [params[6], params[7], 1.0],
        ])
        projected = (H_curr @ pts_1_h.T).T
        projected = projected[:, :2] / projected[:, 2:3]
        return (projected - pts_2).ravel()

    result = nls(residuals, x0, method='lm')
    params = result.x
    H = np.array([
        [params[0], params[1], params[2]],
        [params[3], params[4], params[5]],
        [params[6], params[7], 1.0],
    ])
    ##########################################################################
    #                               END OF YOUR CODE
    ##########################################################################

    return H
```

(d) Apply the computed homography H from both (b) and (c) to the original points. Run the plotting code to visualize all four plots and include them in the notebook. Make sure the destination and transformed points are very close together. Include the visualization in the notebook.

First, implement two functions that converts cartesian coordinates to homogenious coordinates, and the other way around.

In [15]:
```
def to_homog(pts):
    """
    Convert 2D catesian coordinates to homogenious coordinates
    Input:
        pts: (N, 2) numpy array
    Returns:
        new_pts: (N, 3) numpy array
    """

    new_pts = None
```

```
    ##########################################################################
    #                                       TODO
    ##########################################################################
    new_pts = np.hstack([pts, np.ones((pts.shape[0], 1))])
    ##########################################################################
    #                                 END OF YOUR CODE
    ##########################################################################

    return new_pts


def to_cart(pts):
    """
    Convert 2D homogenious coordinates to cartesian coordinates
    Input:
        pts: (N, 3) numpy array
    Returns:
        new_pts: (N, 2) numpy array
    """

    new_pts = None

    ##########################################################################
    #                                       TODO
    ##########################################################################
    new_pts = pts[:, :2] / pts[:, 2:3]
    ##########################################################################
    #                                 END OF YOUR CODE
    ##########################################################################

    return new_pts
```

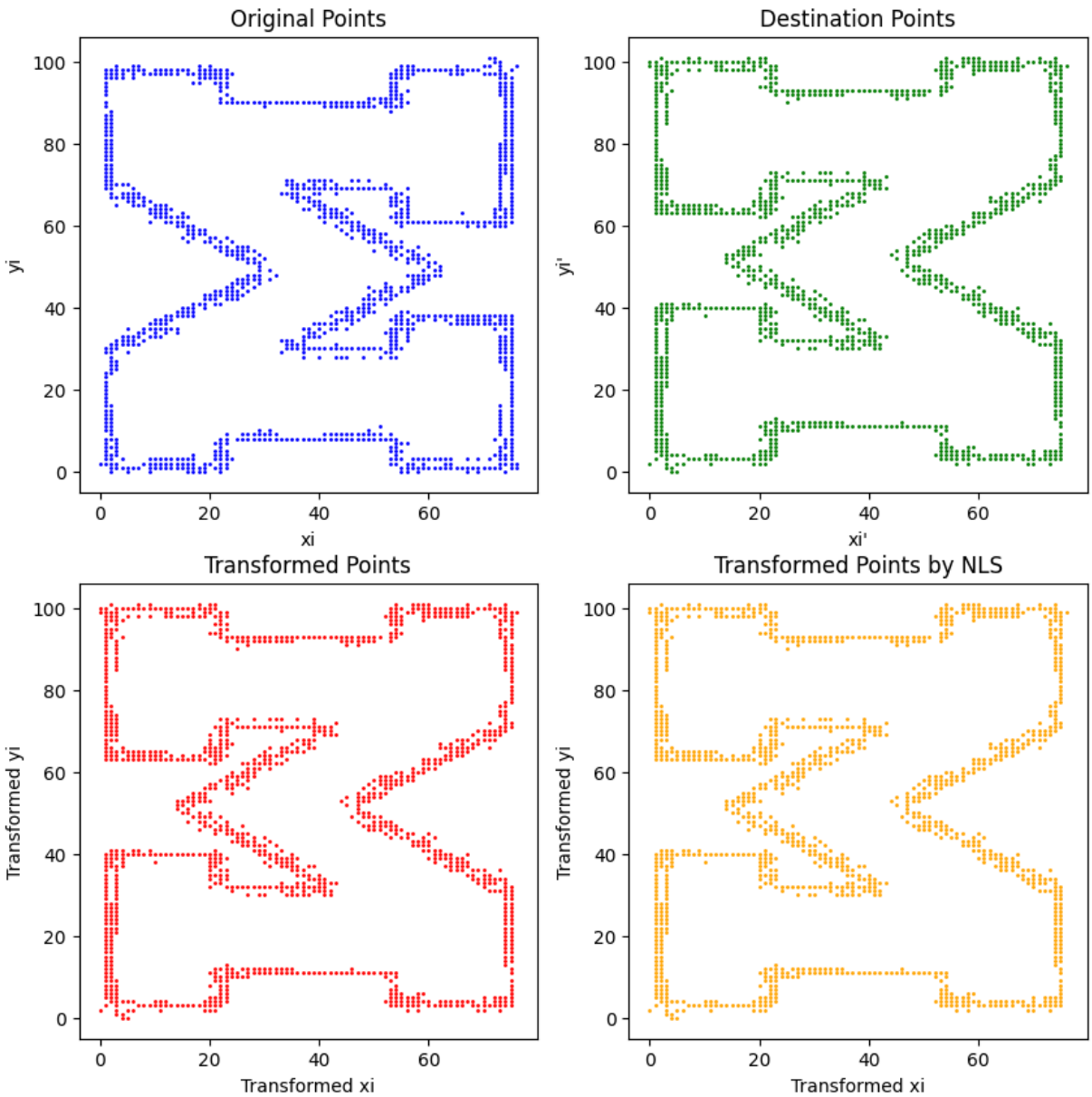
Compute the homography in both ways, apply to the original points, and visualize the results

In [16]:
```
def apply_the_homography(original_points, destination_points):
    """
    Apply the homography to the original points and visualize the results.
    """

    transformed_points = None
    transformed_points_nls = None

    ##########################################################################
    #                                       TODO
    # 1. find the homography matrices both by analytic solution and
    #    nonlinear least squares
    # 2. transform the data to homogeneous coordinates, then apply the
    #    transformation
    # 3. transform the data back to euclidean coordinates
    ##########################################################################
    H = fit_homography(original_points, destination_points)
    H_nls = find_homography_nls(original_points, destination_points)

    original_points_h = to_homog(original_points)
```

```
    transformed_points = to_cart((H @ original_points_h.T).T).T
    transformed_points_nls = to_cart((H_nls @ original_points_h.T).T).T
    ##########################################################################
    #                              END OF YOUR CODE
    ##########################################################################

    fig, ax = plt.subplots(2, 2, figsize=(10, 10))
    ax[0,0].scatter(original_points[:, 0], original_points[:, 1], s=1, c='bl
    ax[0,0].set_title('Original Points')
    ax[0,0].set_xlabel('xi')
    ax[0,0].set_ylabel('yi')
    ax[0,1].scatter(destination_points[:, 0], destination_points[:, 1], s=1,
    ax[0,1].set_title('Destination Points')
    ax[0,1].set_xlabel("xi'")
    ax[0,1].set_ylabel("yi'")
    ax[1,0].scatter(transformed_points[0, :], transformed_points[1, :], s=1,
    ax[1,0].set_title('Transformed Points')
    ax[1,0].set_xlabel('Transformed xi')
    ax[1,0].set_ylabel('Transformed yi')
    ax[1,1].scatter(transformed_points_nls[0, :], transformed_points_nls[1,
    ax[1,1].set_title('Transformed Points by NLS')
    ax[1,1].set_xlabel('Transformed xi')
    ax[1,1].set_ylabel('Transformed yi')
    plt.show()
```

In [17]:

```
apply_the_homography(original_points, destination_points)
```



Problem 2

Next, we will develop an algorithm for stitching a panorama from overlapping photos of the same scene, which amounts to estimating a transformation that aligns one image to another. To do this, we will extract pixel-wise features in both images and match them to obtain correspondences. We will then estimate a homography from these correspondences, and we'll use it to stitch the images together in a common coordinate system.

We provide three example scenes to work on: hill (3 images), wisc (3 images), and hongkong (3 images). We now download and store these examples with a dictionary where the key is the scene name and value is the list containing every image for the scene. Note that we order the images by camera angles from left to right, and each adjacent images contain enough overlap.

In [18]:
```
%%capture
! wget -O images.zip "https://drive.google.com/uc?export=download&id=1CU-jjr
! unzip -n images.zip
```

In [19]:
```
%matplotlib inline
import os
import cv2
import random
import numpy as np
import matplotlib.pyplot as plt
try:
    from google.colab.patches import cv2_imshow
except ImportError:
    def cv2_imshow(img_rgb):
        plt.imshow(img_rgb)
        plt.axis("off")
        plt.show()
```

In [20]:
```
image_dict = {}
for i in os.listdir('images'):
    if not i.endswith('.jpg'):
        continue
    example = i.split('_')[0]
    if example not in image_dict:
        image_dict[example] = []
    image_dict[example].append(i)
image_dict = {k: sorted(v) for k, v in image_dict.items()}
image_dict = {k: [plt.imread('images/'+ v) for v in v] for k, v in image_dic
```

In [21]:
```
def plot_imgs(*imgs):
    fig, ax = plt.subplots(1, len(imgs), figsize=(8, 8*len(imgs)))
    if len(imgs) == 1:
        ax = [ax]
    for a, img in zip(ax, imgs):
        a.set_axis_off()
        a.imshow(img)
    plt.show()

for k, v in image_dict.items():
    print(k)
    plot_imgs(*v)
```

```
wisc
```

hill

hongkong

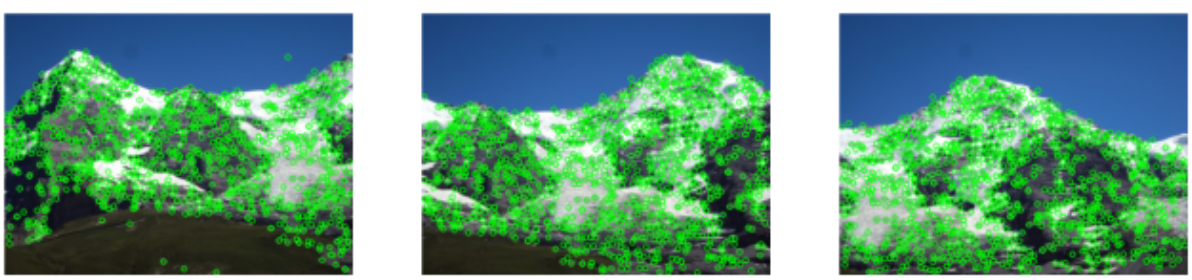
(a) Implement the following function to compute SIFT key points and features for a given image. Run the plotting code to visualize the key points and include them in the notebook.

Hint: you will need `cv2.SIFT_create` and some related functions

In [22]:
```
def get_sift_features(img):
    """
    Compute SIFT features using cv2 library functions.
    Use default parameters when computing the keypoints.

    Input:
        img: cv2 image
    Returns:
        keypoints: a list of cv2 keypoints
        descriptors: a list of SIFT descriptors
    """
```

```
    keypoints = None
    descriptors = None

    ########################################################################
    cache = getattr(get_sift_features, '_cache', {})
    cache_key = id(img)
    if cache_key in cache:
        return cache[cache_key]

    img_for_sift = img
    if img_for_sift.dtype != np.uint8:
        scale = 255.0 if img_for_sift.max() <= 1.0 else 1.0
        img_for_sift = np.clip(img_for_sift * scale, 0, 255).astype(np.uint8
    if img_for_sift.ndim == 3:
        gray = cv2.cvtColor(img_for_sift, cv2.COLOR_RGB2GRAY)
    else:
        gray = img_for_sift

    sift = cv2.SIFT_create()
    keypoints, descriptors = sift.detectAndCompute(gray, None)

    cache[cache_key] = (keypoints, descriptors)
    get_sift_features._cache = cache
    ########################################################################
    #                              END OF YOUR CODE
    ########################################################################

    return keypoints, descriptors
```

In [23]:
```
vis = [cv2.drawKeypoints(i, get_sift_features(i)[0], None, color=(0,255,0),
plot_imgs(*vis)
```



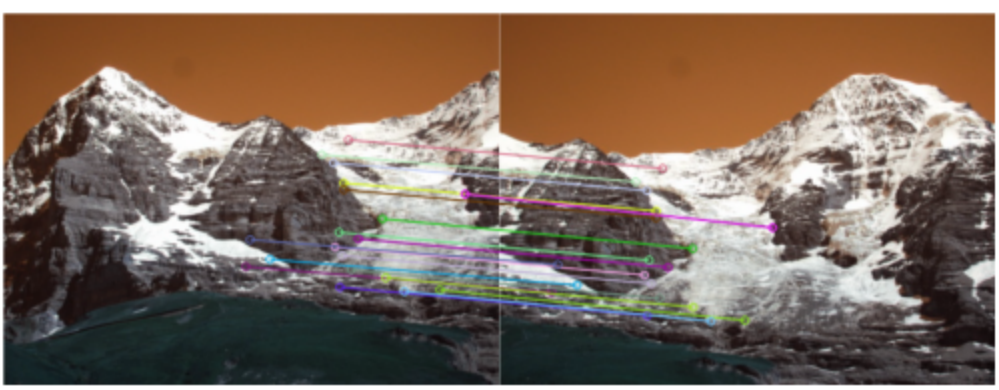
(b) Implement the following function to compute keypoint correspondences between the feature descriptors from two source images using the Lowe's ratio test. Run the plotting code to visualize the resulting correspondences and include them in the notebook.

Hint: you may need to use `cv2.DescriptorMatcher_create`, `knnMatch`, and some related functions

In [24]:
```
def match_keypoints(desc_1, desc_2, ratio=0.75):
    """
    Compute matches between feature descriptors of two images using Lowe's r
```

```
    Input:
        desc_1, desc_2: list of feature descriptors
        ratio (optional): Lowe's ratio test threshold
    Return:
        matches: list of feature matches
    """

    matches = None

    ######################################################################
    if desc_1 is None or desc_2 is None:
        return []

    matcher = cv2.DescriptorMatcher_create(cv2.DescriptorMatcher_BRUTEFORCE)
    knn_matches = matcher.knnMatch(desc_1, desc_2, k=2)

    matches = []
    for pair in knn_matches:
        if len(pair) < 2:
            continue
        m, n = pair
        if m.distance < ratio * n.distance:
            matches.append(m)

    matches = sorted(matches, key=lambda m: m.distance)
    ######################################################################
    #                              END OF YOUR CODE
    ######################################################################

    return matches
```

In [25]:
```
img1, img2 = image_dict['hill'][0], image_dict['hill'][1]
kp_1, desc_1 = get_sift_features(img1)
kp_2, desc_2 = get_sift_features(img2)
matches = match_keypoints(desc_1, desc_2)
match_plot = cv2.drawMatches(img1, kp_1, img2, kp_2, matches[:20], None, fla
cv2_imshow(match_plot[:,:,::-1])
```



(c) Implement the following function that fits a homography using RANSAC, which gives you better robustness to outliers

than `fit_homography(pts1, pts2)`. You are responsible for finding the optimal hyperparameters for RANSAC.

Hint You can call `fit_homography(pts1, pts2)` inside the inner loop of RANSAC. Use the hyperparameters specified in the pdf file.

In [27]:
```
def transform_ransac(x1, x2, verbose=False):
    """
    Implements RANSAC to estimate homography matrix.

    Input:
        x1, x2: (N, 2) matrices
    Return:
        best_model: homography matrix with most inliers
    """

    best_model = None

    ######################################################################
    if x1.shape != x2.shape:
        raise ValueError('x1 and x2 must have the same shape')
    if x1.shape[0] < 4:
        raise ValueError('At least 4 point correspondences are required')

    sample_size = 4
    num_iters = 200
    inlier_threshold = 4.0
    rng = np.random.default_rng(442)
    x1_h = to_homog(x1)
    best_inliers = None
    best_inlier_count = 0
    best_error = np.inf

    def reprojection_errors(H):
        projected_h = (H @ x1_h.T).T
        errors = np.full(x1.shape[0], np.inf, dtype=float)
        valid = np.abs(projected_h[:, 2]) > 1e-8
        if np.any(valid):
            projected = projected_h[valid, :2] / projected_h[valid, 2:3]
            errors[valid] = np.linalg.norm(projected - x2[valid], axis=1)
        return errors

    for _ in range(num_iters):
        idx = rng.choice(x1.shape[0], size=sample_size, replace=False)
        H_candidate = fit_homography(x1[idx], x2[idx])
        errors = reprojection_errors(H_candidate)
        inliers = errors < inlier_threshold
        if np.count_nonzero(inliers) < 4:
            continue

        H_candidate = fit_homography(x1[inliers], x2[inliers])
        errors = reprojection_errors(H_candidate)
        inliers = errors < inlier_threshold
        inlier_count = np.count_nonzero(inliers)
```

```
        if inlier_count < 4:
            continue

        mean_error = errors[inliers].mean()
        if inlier_count > best_inlier_count or (inlier_count == best_inlier_
            best_model = H_candidate
            best_inliers = inliers
            best_inlier_count = inlier_count
            best_error = mean_error

    if best_model is None:
        best_model = fit_homography(x1, x2)
    elif best_inliers is not None and np.count_nonzero(best_inliers) >= 4:
        best_model = fit_homography(x1[best_inliers], x2[best_inliers])

    if verbose:
        print(f'RANSAC inliers: {best_inlier_count}/{x1.shape[0]}')
    ##########################################################################
    #                              END OF YOUR CODE
    ##########################################################################

    return best_model
```

(d) Implement the following function that fits a homography given two input images using your functions from the previous parts.

In [28]:
```
def homography_from_image_pairs(img1, img2):
    """
    Given a pair of overlapping images, generate the homography matrix H.

    Input:
        img1, img2: cv2 images
    Return:
        H: computed homography matrix
    """

    H = None

    ##########################################################################
    cache = getattr(homography_from_image_pairs, '_cache', {})
    cache_key = (id(img1), id(img2))
    if cache_key in cache:
        return cache[cache_key].copy()

    kp_1, desc_1 = get_sift_features(img1)
    kp_2, desc_2 = get_sift_features(img2)
    matches = match_keypoints(desc_1, desc_2)
    if len(matches) < 4:
        raise ValueError('Not enough matches to estimate a homography')

    inlier_threshold = 4.0
    matches_ransac = matches[:150]
    pts_1 = np.float32([kp_1[m.queryIdx].pt for m in matches_ransac])
```

```
    pts_2 = np.float32([kp_2[m.trainIdx].pt for m in matches_ransac])
    H = transform_ransac(pts_2, pts_1)

    pts_1_all = np.float32([kp_1[m.queryIdx].pt for m in matches])
    pts_2_all = np.float32([kp_2[m.trainIdx].pt for m in matches])
    projected_h = (H @ to_homog(pts_2_all).T).T
    errors = np.full(pts_2_all.shape[0], np.inf, dtype=float)
    valid = np.abs(projected_h[:, 2]) > 1e-8
    if np.any(valid):
        projected = projected_h[valid, :2] / projected_h[valid, 2:3]
        errors[valid] = np.linalg.norm(projected - pts_1_all[valid], axis=1)
    inliers = errors < inlier_threshold
    if np.count_nonzero(inliers) >= 4:
        H = fit_homography(pts_2_all[inliers], pts_1_all[inliers])

    cache[cache_key] = H.copy()
    homography_from_image_pairs._cache = cache
    ######################################################################
    #                              END OF YOUR CODE
    ######################################################################

    return H
```

(e) Implement the following function that produces a panorama from a list of overlapping images using your functions from the previous parts. Run the plotting code to visualize the resulting panorama and include them in the notebook.

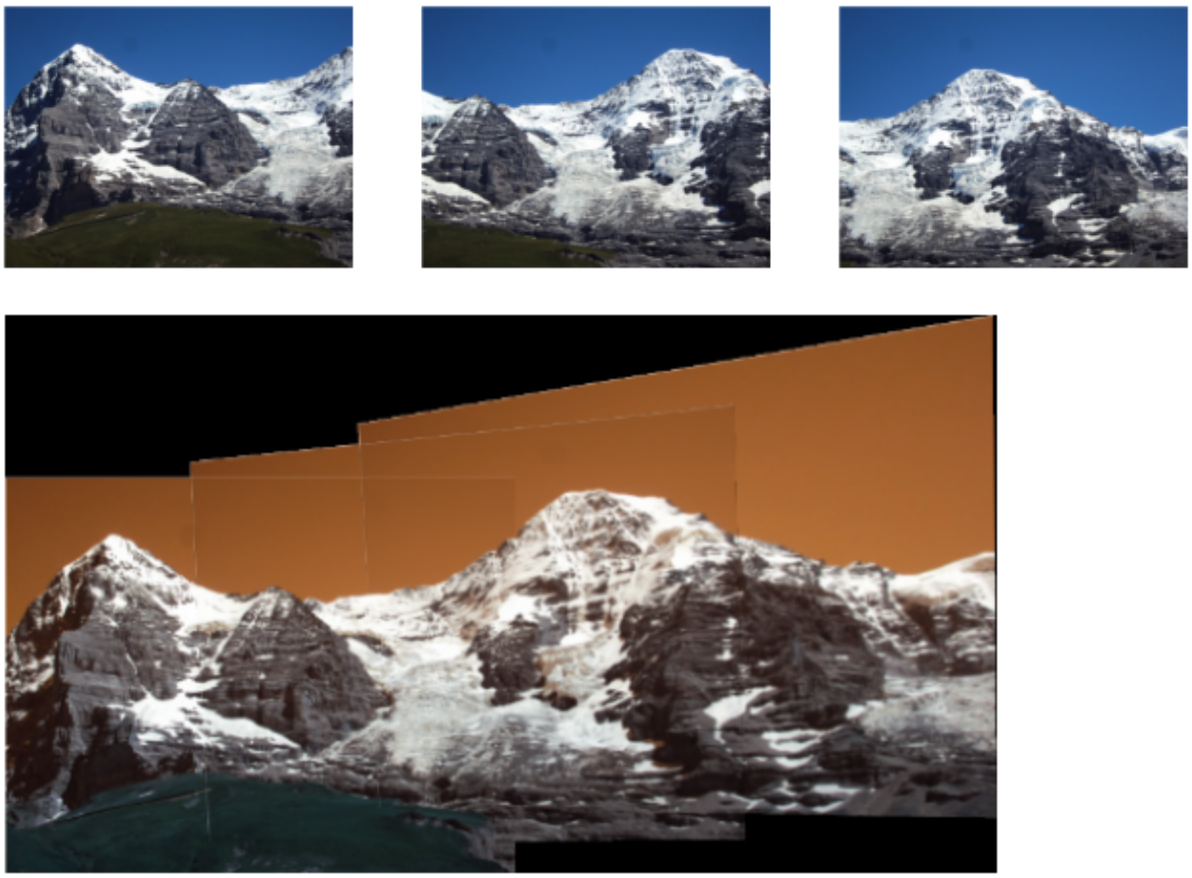
Hints: You may use `cv2.warpPerspective`.

In [29]:
```
def panoramic_stitching(*imgs):
    """
    Given a list of overlapping images, generate a panoramic image.

    Input:
        imgs: cv2 images
    Return:
        final_img: cv2 image of panorama
    """

    final_img = None

    ######################################################################
    if len(imgs) == 0:
        raise ValueError('At least one image is required')
    if len(imgs) == 1:
        return imgs[0].copy()

    pairwise_H = [homography_from_image_pairs(imgs[i], imgs[i + 1]) for i in

    homographies = [np.eye(3)]
    for H_pair in pairwise_H:
        homographies.append(homographies[-1] @ H_pair)
```

```
    all_corners = []
    for img, H in zip(imgs, homographies):
        h, w = img.shape[:2]
        corners = np.array([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]],
        warped_corners = to_cart((H @ to_homog(corners).T).T)
        all_corners.append(warped_corners)

    all_corners = np.vstack(all_corners)
    min_x = int(np.floor(all_corners[:, 0].min()))
    min_y = int(np.floor(all_corners[:, 1].min()))
    max_x = int(np.ceil(all_corners[:, 0].max()))
    max_y = int(np.ceil(all_corners[:, 1].max()))

    translate = np.array([
        [1.0, 0.0, -min_x],
        [0.0, 1.0, -min_y],
        [0.0, 0.0, 1.0],
    ])

    width = max_x - min_x + 1
    height = max_y - min_y + 1

    if imgs[0].ndim == 2:
        accumulator = np.zeros((height, width), dtype=np.float32)
        weights = np.zeros((height, width), dtype=np.float32)
    else:
        accumulator = np.zeros((height, width, imgs[0].shape[2]), dtype=np.f
        weights = np.zeros((height, width, 1), dtype=np.float32)

    for img, H in zip(imgs, homographies):
        warp_H = translate @ H
        warped = cv2.warpPerspective(img.astype(np.float32), warp_H, (width,
        mask = cv2.warpPerspective(np.ones(img.shape[:2], dtype=np.uint8), w

        if imgs[0].ndim == 2:
            mask = (mask > 0).astype(np.float32)
            accumulator += warped * mask
            weights += mask
        else:
            mask = (mask > 0).astype(np.float32)[..., None]
            accumulator += warped * mask
            weights += mask

    final_img = accumulator / np.maximum(weights, 1.0)
    if np.issubdtype(imgs[0].dtype, np.integer):
        final_img = np.clip(final_img, 0, 255).astype(imgs[0].dtype)
    else:
        final_img = final_img.astype(imgs[0].dtype)

    occupied = weights if imgs[0].ndim == 2 else weights[..., 0]
    ys, xs = np.where(occupied > 0)
    if ys.size > 0 and xs.size > 0:
        final_img = final_img[ys.min():ys.max() + 1, xs.min():xs.max() + 1]
    ##########################################################################
    #                               END OF YOUR CODE
```

```
    ##########################################################################

    return final_img
```

In [30]:
```
imgs = image_dict['hill']
plot_imgs(*imgs)
result = panoramic_stitching(*imgs)
cv2_imshow(cv2.resize(result, (480, 270))[:,:,::-1])
```

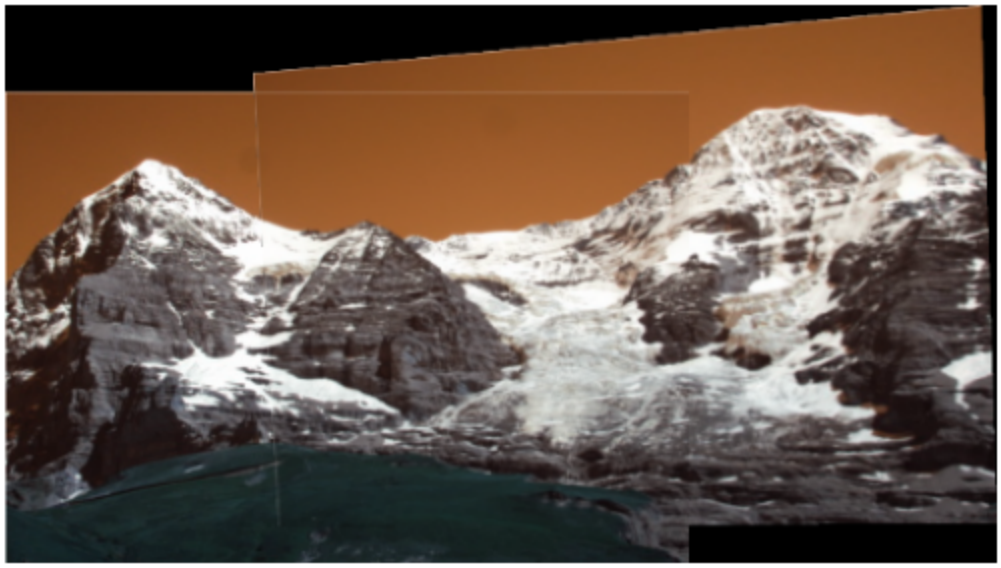


(f) Use the provided function to test your implementations on all three examples (`hill`, `wisc`, and `hongkong`). Save all resulting visualizations in the notebook.

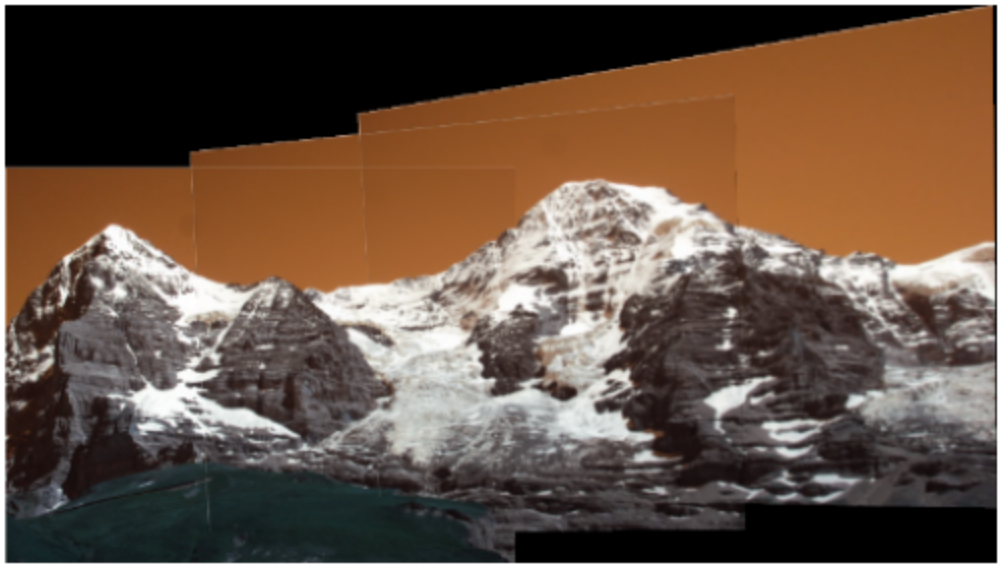
In [31]:
```
k, v = 'hill', image_dict['hill']
print('\n============================ {} ============================\n'.f
plot_imgs(*v)
for i in range(len(v)-1):
    print('{} input images'.format(i+2))
    result = panoramic_stitching(*v[:i+2])
    cv2_imshow(cv2.resize(result, (480, 270))[:,:,::-1])
```

```
============================ hill ============================
```

```
2 input images
```



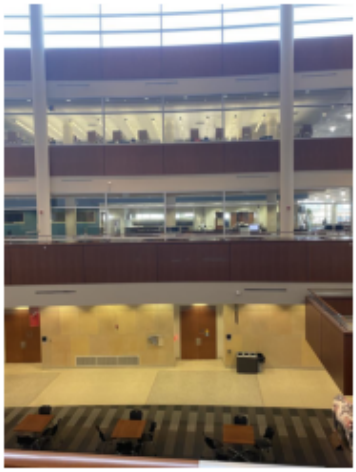
```
3 input images
```



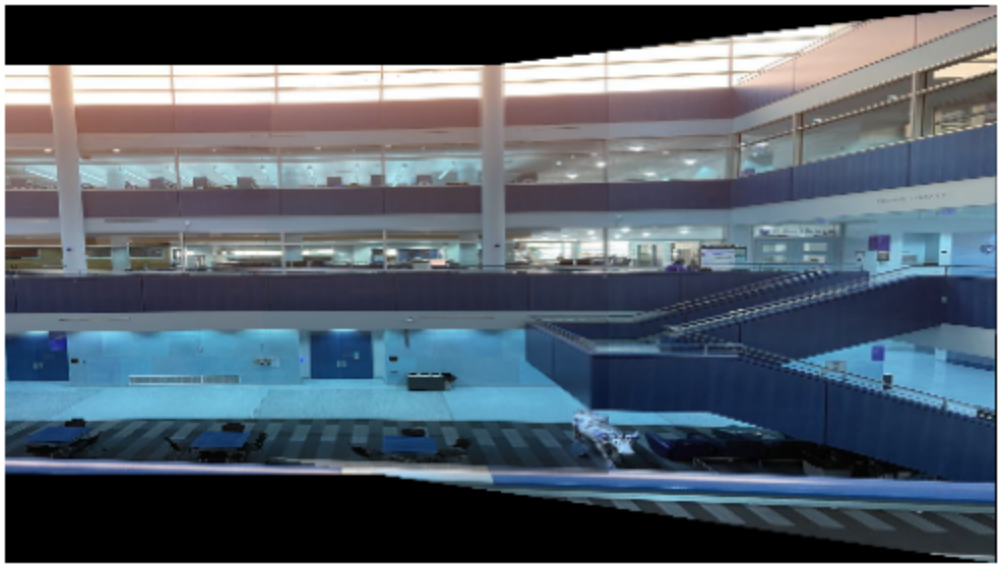
In [32]:

```
k, v = 'wisc', image_dict['wisc']
print('\n============================ {} ============================\n'.f
plot_imgs(*v)
for i in range(len(v)-1):
    print('{} input images'.format(i+2))
    result = panoramic_stitching(*v[:i+2])
    cv2_imshow(cv2.resize(result, (480, 270))[:,:,::-1])
```

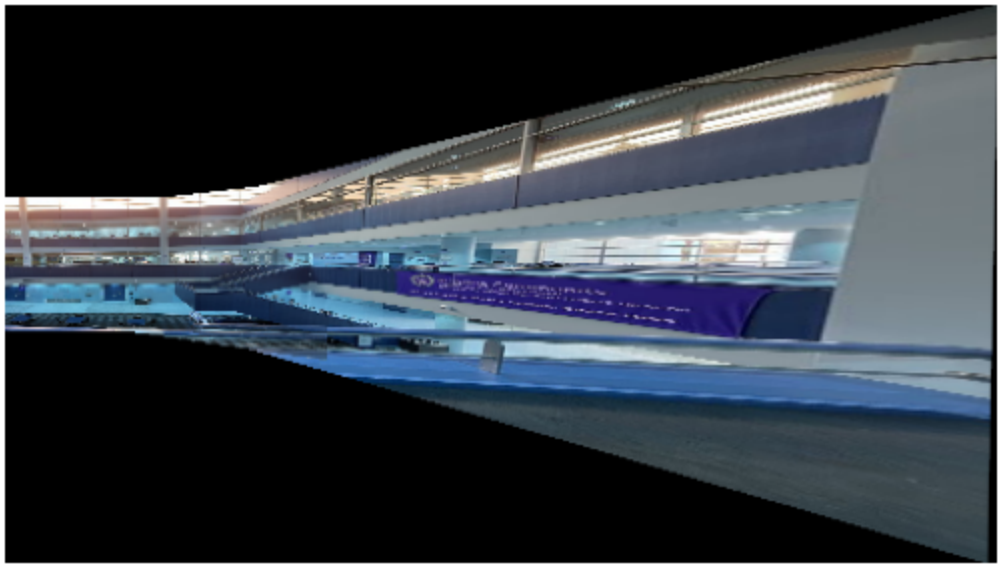
========== wisc ==========



2 input images



3 input images

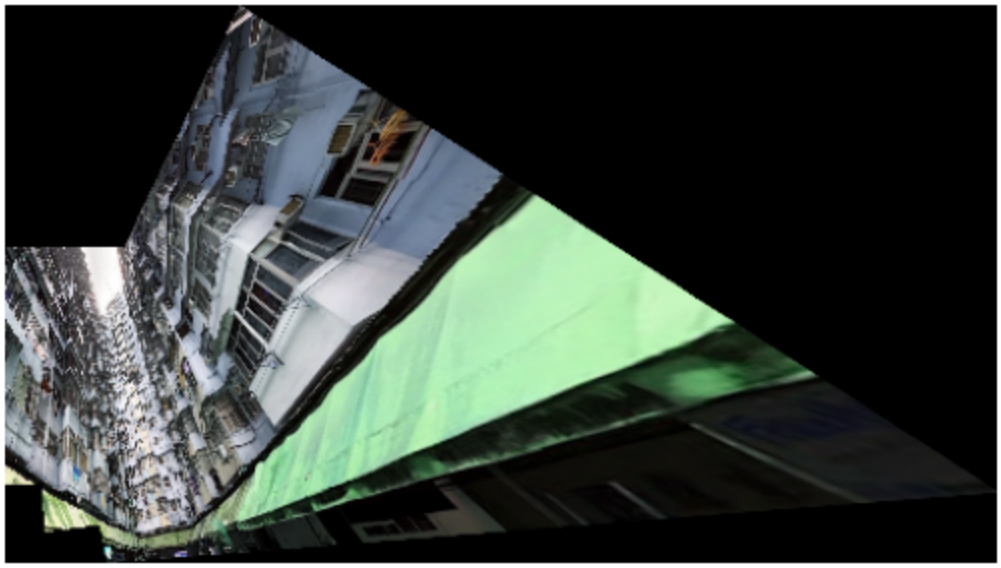


In [33]:
```
k, v = 'hongkong', image_dict['hongkong']
print('\n============================ {} ============================\n'.f
plot_imgs(*v)
for i in range(len(v)-1):
    print('{} input images'.format(i+2))
    result = panoramic_stitching(*v[:i+2])
    cv2_imshow(cv2.resize(result, (480, 270))[:,:,::-1])
```

```
============================ hongkong ============================
```

```
2 input images
```

```
3 input images
```

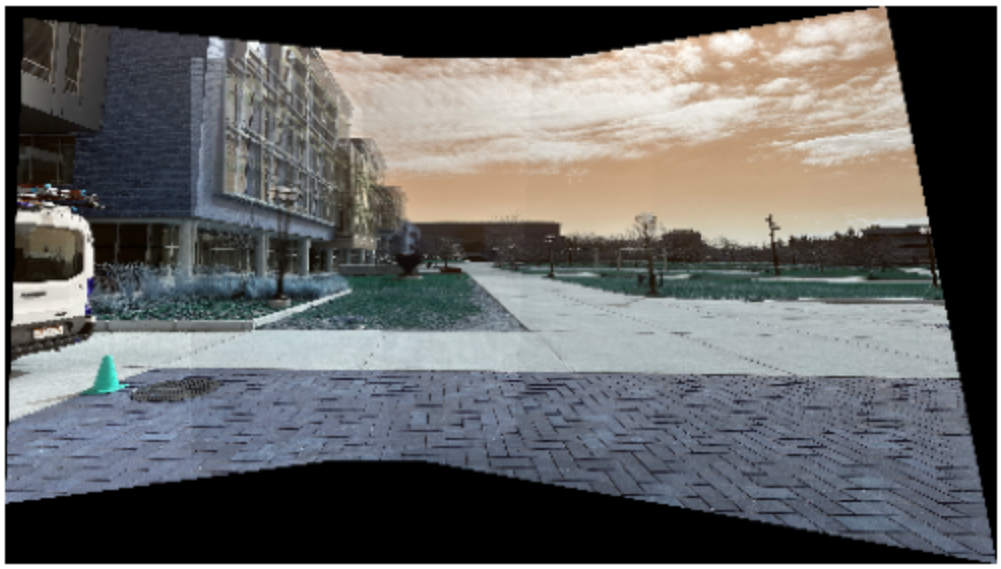


(g) [Optional, Extra Credit] Create your own panorama using images that you capture yourself. **Extend your previous implementation so that the center image serves as the canonical reference frame, and the left and right images are warped into the coordinate frame of the center image**. To receive full credit, display the input images as well as the resulting panorama.

In [36]:

```
    ######################################################################
    #                                   TODO
    # 1. display your images
    # 2. stitch the images together to make your custom panorama
    ######################################################################
left_img = plt.imread('IMG_4188.png')
center_img = plt.imread('IMG_4189.png')
right_img = plt.imread('IMG_4190.png')

plot_imgs(left_img, center_img, right_img)

def panorama_center_ref(left_img, center_img, right_img):
    imgs = (left_img, center_img, right_img)
    H_left_to_center = homography_from_image_pairs(center_img, left_img)
    H_right_to_center = homography_from_image_pairs(center_img, right_img)
    homographies = [H_left_to_center, np.eye(3), H_right_to_center]

    all_corners = []
    for img, H in zip(imgs, homographies):
        h, w = img.shape[:2]
        corners = np.array([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]],
        warped_corners = to_cart((H @ to_homog(corners).T).T)
        all_corners.append(warped_corners)

    all_corners = np.vstack(all_corners)
    min_x = int(np.floor(all_corners[:, 0].min()))
```

```
    min_y = int(np.floor(all_corners[:, 1].min()))
    max_x = int(np.ceil(all_corners[:, 0].max()))
    max_y = int(np.ceil(all_corners[:, 1].max()))

    translate = np.array([
        [1.0, 0.0, -min_x],
        [0.0, 1.0, -min_y],
        [0.0, 0.0, 1.0],
    ])

    width = max_x - min_x + 1
    height = max_y - min_y + 1

    if imgs[0].ndim == 2:
        accumulator = np.zeros((height, width), dtype=np.float32)
        weights = np.zeros((height, width), dtype=np.float32)
    else:
        accumulator = np.zeros((height, width, imgs[0].shape[2]), dtype=np.f
        weights = np.zeros((height, width, 1), dtype=np.float32)

    for img, H in zip(imgs, homographies):
        warp_H = translate @ H
        warped = cv2.warpPerspective(img.astype(np.float32), warp_H, (width,
        mask = cv2.warpPerspective(np.ones(img.shape[:2], dtype=np.uint8), w

        if imgs[0].ndim == 2:
            mask = (mask > 0).astype(np.float32)
            accumulator += warped * mask
            weights += mask
        else:
            mask = (mask > 0).astype(np.float32)[..., None]
            accumulator += warped * mask
            weights += mask

    final_img = accumulator / np.maximum(weights, 1.0)
    if np.issubdtype(imgs[0].dtype, np.integer):
        final_img = np.clip(final_img, 0, 255).astype(imgs[0].dtype)
    else:
        final_img = final_img.astype(imgs[0].dtype)

    occupied = weights if imgs[0].ndim == 2 else weights[..., 0]
    ys, xs = np.where(occupied > 0)
    if ys.size > 0 and xs.size > 0:
        final_img = final_img[ys.min():ys.max() + 1, xs.min():xs.max() + 1]

    return final_img

pano = panorama_center_ref(left_img, center_img, right_img)
cv2_imshow(cv2.resize(pano, (480, 270))[:, :, ::-1])

    ##########################################################################
    #                              END OF YOUR CODE
    ##########################################################################
```



Convert Notebook to PDF

Now, save your notebook as PDF and submit to gradescope. You can use `Ctrl/Cmd + P` or the following code, but make sure the pdf contains **BOTH your code and all required visualizations**.

In [ ]:
```
# # Generate pdf
# # Please provide the full path of the notebook file below
# # Important: make sure that your file name does not contain spaces!
# import os
# notebookpath = '' # Ex: notebookpath = '/content/drive/My Drive/Colab Note
# drive_mount_point = '/content/drive/'
# from google.colab import drive
# drive.mount(drive_mount_point)
# file_name = notebookpath.split('/')[-1]
# get_ipython().system("apt update && apt install texlive-xetex texlive-font
# get_ipython().system("pip install pypandoc")
# get_ipython().system("apt-get install texlive texlive-xetex texlive-latex-
# get_ipython().system("jupyter nbconvert --to PDF {}".format(notebookpath.r
```

```
# from google.colab import files
# files.download(notebookpath.split('.')[0]+'.pdf')
```

In [ ]: